Limits of type-level transition guarantees in composable state machines: a closed-loop case study in Haskell

PETERSON CARARA JUNIOR
Faculdade de Tecnologia da Baixada Santista "Rubens Lara" (Fatec BS), Brazil
(email: `[e-mail to be added]`*)*

Abstract

Typed state-machine libraries promise that illegal transitions do not compile. We ask what that guarantee is worth once machines are composed. Using the `crem` library for Haskell, we study a four-state avionics machine composed with a battery machine and a reactive environment closed by `Feedback`. The domain is an illustrative discrete case study; we make no claim about real avionics. A rejection matrix over all 16 vertex pairs and nine mutants of the implementation show that the checker rejects edges outside the topology and omitted cases written without a wildcard, but accepts a wrong destination on an allowed edge, swapped events, wrong outputs, faulty environments and an omission hidden by a catch-all clause, which only tests catch. We then describe a validation method that other `crem` users can reuse: a reviewable transition table with the provenance of each rule, a table-driven oracle, a typed boundary adapter, property-based testing with Hedgehog, and an exhaustive exploration of the closed-loop state graph that runs the real `crem` machines from any triple of vertices. Under a stated abstraction hypothesis, the exploration establishes equivalence between implementation and oracle for sequences of any length, and exposes a global defect that every edge-level check misses: the environment reacts to the *edge* "the battery became critical" and not to the *level*, so the aircraft can stay airborne with a critical battery forever. A TLA+ model checked with TLC independently reproduces the finding. The defect belongs to the environment model, not to `crem`; the lesson is where the type-level guarantee stops.

1 Introduction

Finite-state machines are a standard way to specify reactive behaviour, and a long line of work asks how to make a programming language enforce the specification. In a pure functional language the machine becomes a function from state and event to a new state, algebraic data types make the sets of states and events closed, and the compiler checks that the transition function covers every pair [5, 7]. Libraries such as `motor` [11] and `crem` [8] go further: they lift the allowed transitions to the type level, so that an action that returns a destination not permitted from its origin fails to type-check.

This is a real guarantee, and it is local. It says that *each edge* used by the code is in a declared topology. It does not say which event causes which edge, and it says nothing about what happens when several machines are composed and fed back to one another. Perone and Karachalias themselves list, among the limitations of `crem`, that the topology is not tied to the inputs that trigger the transitions [8]. The practical question is therefore: *what does an engineer still have to verify after the compiler is satisfied, and how can that be done in the same language?*



© 2026 Copyright held by the owner/author(s).
This work is licensed under a Creative Commons Attribution 4.0 International License.

We answer it with a small, fully worked case study and a method.

Case study. A four-state avionics machine (`Grounded`, `InFlight`, `EmergencyLanding`, `Shutdown`), a three-level battery machine, and a reactive environment that observes their outputs and feeds events back one tick later. The system is built with `crem`'s `Parallel` and `Feedback`. It is an illustration chosen to exercise composition; there are no real flight-safety requirements behind it, and we make no claim about real avionics.

Contributions.

(1) A precise delimitation of what the `crem` type checker enforces in this case study (Sect. 4): a rejection matrix over all 16 pairs of vertices with the *cause* of each rejection checked, and nine mutants of the implementation, two refused by the compiler and seven accepted and killed only by tests.
(2) A reusable validation method (Sect. 5): a reviewable transition table that records, for each rule, whether it comes from the requirements, from a project convention or from a modelling choice; an oracle driven by that table; a typed adapter for external input; and property-based testing with reproducible seeds and shrinking.
(3) An exhaustive exploration of the closed-loop state graph that executes the *real* `crem` machines from an arbitrary triple of vertices (Sect. 5.5). Under a stated hypothesis, it replaces bounded enumeration by a statement about sequences of any length.
(4) A finding that no edge-level check can make: a reachable blind state in which the aircraft stays airborne with a critical battery forever (Sect. 6), reproduced independently by a TLA+ model checked with TLC.

What this paper does not claim. It introduces no new type system, no formal semantics and no general proof that composition preserves the guarantees of its parts. The environment is a modelling choice of the author, and the defect we report is a property of that model, not of `crem` or of GHC. The table, the oracle and the TLA+ specification were written by the same person who wrote the implementation: the independence we obtain is of *code*, not of *authorship* (Sect. 7.3).

Section 2 reviews `crem`, Sect. 3 introduces the case study, Sect. 4 delimits the guarantee, Sect. 5 presents the method, Sect. 6 the results, Sect. 7 the lessons and threats, and Sect. 8 related work.

2 Background: topologies and composition in crem

A `crem` machine is a Mealy machine: on each input it emits an output and updates its state. The vertices of the state graph are promoted to the type level with the `singletons` library [4], and the *topology* is a value listing, for each vertex, the vertices that may follow it. Listing 1 shows the avionics machine. The action receives the current vertex as a singleton and returns an `ActionResult` built by `pureResult`, which demands the constraint `AllowedTransition topology from to`.

How the constraint is solved. In `crem` 0.1.0.0, `AllowedTransition` has four instances, all marked `INCOHERENT`: one succeeds when the edge is the first in the origin's list, one skips the first edge of the origin, one skips the first vertex of the topology, and one for the identity edge. Resolution is thus a search for any path that closes the proof, and a missing edge leaves a residual constraint over the empty list. The identity edge is always allowed, so an event that

Listing 1. Topology and action of the avionics machine (excerpt).

```
$(singletons [d|
  data AvionicsVertex = Grounded | InFlight | EmergencyLanding | Shutdown
  avionicsTopology :: Topology AvionicsVertex
  avionicsTopology = Topology
    [ (Grounded,         [InFlight, Shutdown])
    , (InFlight,         [EmergencyLanding])
    , (EmergencyLanding, [Grounded]) ] |])

avionicsAction :: Applicative m ⇒ SAvionicsVertex v → AvEvent
               → ActionResult m AvionicsTopology SAvionicsVertex v Output
avionicsAction SGrounded TakeoffSignal   = pureResult MsgTakeoff   SInFlight
avionicsAction SInFlight CriticalBattery = pureResult MsgEmergency SEmergencyLanding
-- ... 16 equations in total, none with a wildcard
```

Table 1. Transition function of the avionics machine. R: stated in the requirement text; unmarked cells keep the state (project convention) and output `MsgIgnored`.

State	TakeoffSignal	CriticalBattery	LandedSignal	SystemCutoff
Grounded	InFlight (R)	–	–	Shutdown (R)
InFlight	–	EmergencyLanding (R)	–	–
EmergencyLanding	–	–	Grounded (R)	–
Shutdown	–	–	–	–

must not change the state is written by returning the same vertex. We did not prove that this resolution is correct for every topology; for ours it agrees with the specification on all 16 pairs (Sect. 4).

Composition. Machines are the leaves (`Basic`) of a tree whose nodes are `Sequential`, `Parallel`, `Alternative`, `Feedback` and `Kleisli`. Each leaf keeps its own state and topology. `Feedback` runs the forward machine on an input, passes each output to the return machine, and re-injects every input that the return machine emits, recursively, until it emits nothing. This makes `Feedback` *instantaneous* within one call of `run`; any delay has to be modelled, as we do below.

3 The case study

The avionics machine. Table 1 gives its transition function. Only four rows come from the requirement text (marked R); the other twelve pairs are a project convention (the state does not change and the output is `MsgIgnored`), because the text does not say what happens there. The output names are also a convention. The one prohibition in the requirements is the direct edge from `InFlight` to `Shutdown`.

The battery machine. Three levels (`Charged`, `Low`, `Critical`); `Drain` goes one level down, `Recharge` one up, and `ShedLoad` (a load shedding that only relieves `Critical` to `Low`) was added for the closed loop. There is no edge from `Charged` to `Critical`. Every rule of this machine is a modelling choice.

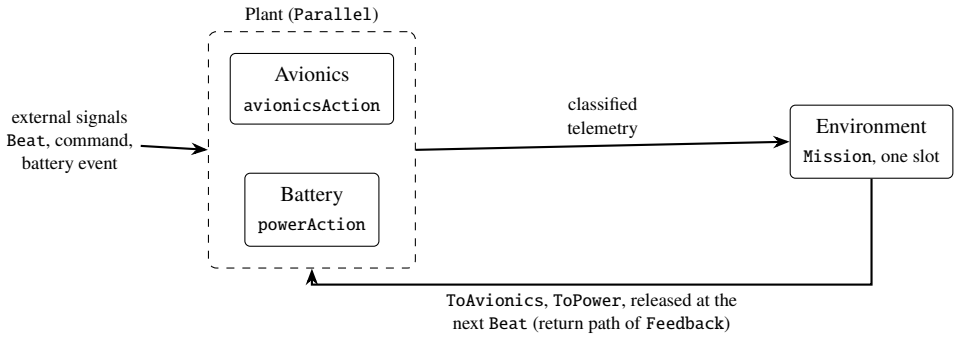


Fig. 1. The closed loop. The environment holds relevant telemetry and releases it at the next tick.

The environment and the loop. The plant is the `Parallel` composition of the two machines. A third machine, `Mission`, observes their telemetry and closes the loop with `Feedback` (Fig. 1). Because `Feedback` is instantaneous, the one-tick delay is obtained by making `Mission` a *one-slot register*: relevant telemetry makes it *hold* an event, and the `Beat` that opens the next tick makes it *release* the held event. Concretely, a critical battery is held as `CriticalHeld` and released as `CriticalBattery` to the avionics; `MsgEmergency` is held as `TouchdownHeld` and released as `ShedLoad` to the battery followed by `LandedSignal`; `MsgLanded` is held as `CutoffHeld` and released as `SystemCutoff`. While the slot is occupied, new telemetry is *dropped*. A tick processes the `Beat`, then the command, then the battery event, and every signal released during a tick is processed, depth first, before the next external signal.

Listing 2 shows three of the 20 equations of `Mission`; the last one is the one that matters later.

Listing 2. Excerpt of the environment machine.

```
missionAction SStandby CueBatteryCritical
  = pureResult [] SCriticalHeld
missionAction SCriticalHeld CueBeat
  = pureResult [ToAvionics CriticalBattery] SStandby
missionAction SCriticalHeld CueBatteryCritical
  = pureResult [] SCriticalHeld   -- slot busy: dropped
```

Three environment policies. `OneSlot` is the implementation above. `Queue` is a loss-free variant in which each kind of held event enters once. `Level` is `OneSlot` plus a reaction to the battery *level*: on each `Beat`, with the slot free, the battery in `Critical` and the avionics `InFlight`, it releases `CriticalBattery`. Only `OneSlot` exists in `crem`; the other two exist in the oracle.

4 What the type checker does and does not guarantee

4.1 Rejection matrix

For each of the 16 ordered pairs of vertices we generate a minimal module whose action returns the destination from the origin, and compile it. The expected outcome is derived from the transition table, not from a hand-written list: the four edges and the four identities

Table 2. Compilation of `pureResult` for every (origin, destination) pair. ✓: compiles (identity on the diagonal); ×: rejected with the expected cause.

origin \ destination	`Grounded`	`InFlight`	`EmergencyLanding`	`Shutdown`
`Grounded`	✓	✓	×	✓
`InFlight`	×	✓	✓	×
`EmergencyLanding`	✓	×	✓	×
`Shutdown`	×	×	×	✓

Table 3. Mutants of the `crem` implementation. "Accepted" means that the compiler accepts the mutant.

Id	Defect	Compiler	Killed by
H1	illegal edge `InFlight`→`Shutdown` in topology and action	accepted	all five suites
H2	wrong destination on an allowed edge (`Grounded`, `TakeoffSignal` → `Shutdown`)	accepted	all five
H3	swapped events in `Grounded` (`TakeoffSignal`, `SystemCutoff`)	accepted	all five
H4	illegal action *without* changing the topology	**rejected**	–
H5	omitted (state, event) pair	**rejected** (exhaustiveness as error)	–
H6	wrong output (`EmergencyLanding`, `LandedSignal` emits `MsgIgnored`)	accepted	four suites
H7	environment: `CriticalHeld` releases `LandedSignal`	accepted	three suites
H8	battery: `Low` with `Drain` stays `Low`	accepted	four suites
H9	omitted pair (`Grounded`, `TakeoffSignal`) plus a final catch-all clause returning the same vertex	accepted	all five

compile, the other eight pairs must be rejected. A pair counts as rejected only if GHC reports exactly one error and that error is `Could not deduce AllowedTransition` naming the pair; any other failure is a mismatch. All 16 results agree with the table (Table 2).

This matrix checks the *topology*. It is not the same as the origin-by-event matrix, which also has 16 cells but checks that the transition function covers every pair. That second property is, in GHC, a warning (`-Wincomplete-patterns`) and only becomes a compile error if the project asks for it; we compile with `-Werror=incomplete-patterns` and write all 16 equations without a wildcard, since a wildcard would hide an omission (mutant H9 below).

4.2 Mutation of the implementation

To learn which defects the checker *cannot* see, we injected nine defects, one at a time, into a temporary copy of the code and ran every test suite on each (Table 3).

No mutant survived. But only two were stopped by the compiler: the illegal edge when the topology is left alone (H4) and the omitted pair (H5). When the defect is in the topology as well (H1), the compiler cannot tell it from the specification; a wrong destination on an allowed edge (H2), swapped events (H3), wrong outputs (H6) and faulty environment or battery rules (H7, H8) are all accepted. These are exactly the *event–destination* gap: the type fixes which vertices may follow which, not which event leads where. The omission is caught

only while it is written without a wildcard: with a final catch-all clause that returns the same vertex (H9), the identity edge is always allowed, the clause type-checks for every vertex, and the omission compiles silently. Mutants were chosen by the author from the defect classes that a technical review of the project asked for (illegal edges, wrong destinations, swapped events), plus omission, catch-all, output, environment and battery; they are not a systematic sample of faults.

5 A validation method for typed state machines

The method has three levels. Level 1 is the static checks above. Levels 2 and 3 are what the compiler does not give.

5.1 A reviewable table with provenance

The transition functions are written as CSV files (16 rows for the avionics machine, 9 for the battery), each row with an *origin*: `REQ` (the rule is in the requirement text), `CONV` (a project convention for a pair the text does not treat) or `MODEL` (a modelling choice with no requirement behind it). In our case study 4 rows are `REQ`, 12 are `CONV`, and all 9 battery rows are `MODEL`. The table is the artefact a reviewer reads, and it lists the open questions explicitly (for example, whether `InFlight` with `SystemCutoff` should be ignored or explicitly rejected). The loader checks that each (state, event) pair has exactly one row, and its own test suite checks that this validator can fail (a missing row, a repeated row, an unknown name).

5.2 A table-driven oracle

A reference model of the closed loop, in plain Haskell and without `crem`, reads the tables and implements the environment policies. It deliberately does not call the project's transition function: it shares only the data types. Comparing it with the `crem` implementation is therefore a comparison between two *programs*, one of which is driven by a reviewable document. Both are reduced to the same list of *step summaries* (event received, output, resulting vertices, environment vertex), one per signal processed, and compared step by step; the comparator reports the first divergence.

5.3 A typed boundary

External input (generated scenarios, an oracle in another language) is untyped text. A small adapter decodes it with a *strict* vocabulary into the `AvEvent` and `PowerEvent` types, returning `Left` with the line number on any other input:

Listing 3. The typed boundary (core).

```
decodeEvent :: String → Either BoundaryError AvEvent
decodeEvent "TAKEOFF"  = Right TakeoffSignal
decodeEvent "CRITICAL" = Right CriticalBattery
decodeEvent "LANDED"   = Right LandedSignal
decodeEvent "CUTOFF"   = Right SystemCutoff
decodeEvent other      = Left (UnknownToken other)
```

The adapter guarantees that every value that crosses it is a valid constructor. It does *not* guarantee that the event leads to the right destination; that is the job of the table. Its contract

is tested with 21 checks, including 13 hostile strings that must never raise an exception, and with three Hedgehog properties (round trip, totality, strictness).

5.4 Property-based testing

With Hedgehog [10] (in the tradition of QuickCheck [3]) the property is "for every list of ticks, the `crem` trace equals the oracle's trace". Lists have up to 40 ticks, against 4 in the earlier exhaustive enumeration; the seed is fixed and printed, so that two runs produce the same log, and shrinking reduces each counterexample to a local minimum. The system is a pure Mealy machine with no opaque state to be referenced across commands, so we did not need Hedgehog's state-machine API.

5.5 Exhaustive exploration of the closed loop

Bounded enumeration (we first ran 168 421 scenarios of up to 4 ticks over the full alphabet of 20 per-tick inputs, and 87 381 scenarios of up to 8 ticks over a reduced alphabet of four) says nothing about longer sequences. But the state of the closed loop between two ticks is just the triple of vertices (avionics, battery, environment): at most $4 \times 3 \times 4 = 48$ values. We can therefore explore the reachable graph to its closure.

The key point is to explore the *implementation*, not a re-description of it. `closedLoop` is a function of three singletons; with `withSomeSing` we can build the real `crem` machine, with its `Feedback`, its `Mission` and its actions, at any triple and run one tick on it (Listing 4).

Listing 4. One tick of the real `crem` loop from an arbitrary triple of vertices.

```
cremNext :: AbsState → TickInput → (AbsState, [StepSum])
cremNext (AbsState a p q) inp =
  withSomeSing @AvionicsVertex (avVs !! a) $ \sa →
    withSomeSing @PowerVertex (pwVs !! p) $ \sp →
      withSomeSing @MissionVertex (msVs !! q) $ \sq →
        summarize (runAll (closedLoop sa sp sq) (externalSignals inp))
```

The abstraction hypothesis. The exploration is faithful if the future behaviour of the loop depends only on the triple. This holds by construction (a `Basic` machine's hidden state is the singleton of its vertex, and `closedLoop` is a function of the three vertices), but it is a hypothesis about `crem`'s semantics, not a theorem we proved. We check it empirically: for the 8 421 scenarios of up to 3 ticks, iterating `cremNext` gives exactly the same steps and final triple as the ordinary scenario simulator.

5.6 A TLA+ specification

An independent specification of the same loop was written in TLA+ [6] from the tables, without reading the Haskell code. Each signal is one step; the signals of a tick are a queue, and what the environment releases is pushed to the *front* of the queue, which reproduces the depth-first order of `Feedback`. Weak fairness is assumed for the processing of signals and for the occurrence of ticks, and the inputs of each tick are free (including idle forever). The properties are the topology safety property, the event–destination property for `TakeoffSignal`, termination of the feedback within a tick, and `NoPermanentBlindness`: infinitely often, the avionics is not airborne with a critical battery.

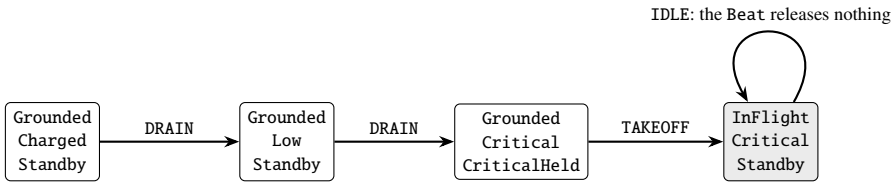


Fig. 2. The shortest path to a blind state under `OneSlot`, written as (avionics, battery, environment). Idle ticks leave the state unchanged forever.

6 Results

All results come from recorded runs with GHC 9.6.7, `crem` 0.1.0.0, Hedgehog 1.7 and TLC 2.19 (TLA+ Tools 1.7.4). A clean build of all targets produces no GHC warnings; the reproduction script checks this on every run.

6.1 Equivalence for sequences of any length

Exploring the closed loop from `(Grounded, Charged, Standby)` with the 20 per-tick inputs closes at **26 reachable triples (of 48) and 520 edges**. With the `crem` machines run at every triple, three statements hold for sequences of any length, under the abstraction hypothesis:

- no step of any edge violates topology safety (no step from `InFlight` to `Shutdown`; `Shutdown` only from `Grounded`; `EmergencyLanding` only from `InFlight`);
- the `crem` loop and the oracle for `OneSlot` have the same reachable states and the same edges, with the same steps (event, output, vertices, environment vertex) on every edge;
- the feedback terminates on all 520 (state, tick) combinations, with at most 5 steps per tick (the `Beat`, up to two releases, the command and the battery event).

6.2 The blind state

With `OneSlot`, two reachable triples have the avionics airborne and the battery `Critical` from which idle ticks never lead out: `(InFlight, Critical, Standby)` and `(InFlight, Critical, CutoffHeld)`, each reached in 3 ticks. The first is reached by `DRAIN`, `DRAIN`, `TAKEOFF` (Fig. 2): the battery becomes critical on the ground, the held event is released while the avionics is `Grounded`, which ignores `CriticalBattery`, and the take-off then happens with the battery already critical. Nothing triggers the emergency, because the environment reacts to the *entry* of the battery into `Critical`, not to its *level*. The second is a different route to the same effect: the critical-battery news arrives while the slot is occupied by a landing and is dropped.

Table 4 compares the three policies. The property is checked over the whole graph: a state is *forever blind* if some infinite path with arbitrary inputs stays airborne with a critical battery (the greatest fixed point of "blind and having a blind successor").

A loss-free queue does not remove the defect; reacting to the level does, for every sequence and not only up to a bound. This turns an earlier bounded measurement (172 scenarios of up to 8 ticks over a reduced alphabet end in this situation under `OneSlot`, 0 under `Level`) into a statement about the whole graph. The smallest scenario on which the `crem` loop diverges step by step from `Level` has four ticks: `DRAIN`, `DRAIN`, `TAKEOFF`, `IDLE`. With the recorded

Table 4. Environment policies over the closed-loop graph (20 inputs per tick).

Policy	reachable states	blind under idle ticks	forever blind (any inputs)
`OneSlot` (implemented in `crem`)	26	2	2
`Queue` (oracle only)	28	1	1
`Level` (oracle only)	26	0	0

seed, the Hedgehog shrinker, started from scenarios of up to 40 ticks, also reached these four ticks; but shrinking only guarantees a local minimum, so it is the breadth-first search, not the shrinker, that establishes that no shorter scenario exists.

6.3 A correction to a bounded conclusion

The earlier bounded runs reported that dropping events when the slot is busy never changed the final state (0 of 87 381 scenarios differing between `OneSlot` and `Queue`). That alphabet had no `LandedSignal` command and no tick combining a command with a battery event. Over the full alphabet, the product of the two policies has 282 reachable pairs, and in **232** of them the (avionics, battery) pair differs at the end of a tick. The smallest case has four ticks: `DRAIN`; `TAKEOFF DRAIN`; `LANDED`; `RECHARGE`. It ends with the avionics `Grounded` under `OneSlot`, which dropped the landing news while its slot held the touchdown, and `Shutdown` under `Queue`, which kept it. The earlier statement was true of the alphabet it was measured on; it was not a property of the model. This is the kind of conclusion that bounded enumeration over a reduced alphabet cannot support, and that closing the graph can.

6.4 Property-based tests and mutants

Hedgehog found the `crem` trace equal to the oracle's in 5 000 random closed-loop scenarios of up to 40 ticks and 3 000 open-loop ones, and the trace properties of the original suite held in 3 000 scenarios of up to 60 ticks. To check that the test can fail, four defects were injected into the oracle's table, which is equivalent to a defect in the implementation because the comparison is symmetric, and the oracle was also run with the `Level` policy against `crem`. All five were detected and shrunk: a wrong destination (1 tick), an illegal edge (2), swapped events (1), a wrong output (3) and the `Level` policy against `crem` (4).

6.5 Model checking

TLC closed the TLA+ model with 1 143 distinct states (1 638 generated). With `OneSlot`, it reports a violation of `NoPermanentBlindness` with a lasso counterexample that is the path of Fig. 2; without that property every other property holds. With `Level`, all properties hold. The set of triples with an empty queue has 26 elements and is *identical* to the set reached by the Haskell exploration. Two independent routes, the `crem` machines run at every triple and a TLA+ specification written from the tables, therefore reach the same state space and the same verdict on the battery policy. This lowers the risk of an error in one of the two; it does not lower the risk of an interpretation error common to both, since the same person wrote them. TLC checks the model, not the Haskell code.

Table 5. What each kind of evidence establishes in this case study.

Evidence	Establishes	Does not establish
Compiler (topology, exhaustiveness as error)	edges outside the topology and omitted pairs are rejected	right destination for an event; any sequence property
Table and oracle, exhaustive on pairs	implementation conforms to the table on all 16 and 9 pairs	that the table reflects real requirements
Closure of the closed-loop graph	equivalence and safety for any length, under the abstraction hypothesis; the blind state	correctness of `crem`; real avionics
Property-based tests	divergences in random scenarios, with seeds and shrinking	absence of divergence outside the sample
TLC	the same verdicts on an independently written model	anything about the Haskell code itself

6.6 What each kind of evidence establishes

Table 5 summarises what each kind of evidence in this section does and does not establish; the right-hand column is as important as the left.

7 Discussion

7.1 Lessons for `crem` users

Treat the type as a per-edge guarantee. In our mutation experiment the compiler caught two of nine defects. Writing every equation without a wildcard and compiling exhaustiveness as an error is what makes one of the two work; H9 shows that a catch-all clause undoes it. The wrong destination on an allowed edge is the most dangerous because it is silent; the cheap defence is a table in which events appear, checked exhaustively against the actions.

Keep the provenance of each rule. Of the 16 rows of the avionics table only four come from the requirements. The distinction between `REQ`, `CONV` and `MODEL` turned a vague worry (“is the specification right?”) into a short list of questions for a reviewer.

Close the graph instead of enumerating strings. The state space between ticks was tiny. Exploring it to closure with the real machines took seconds, replaced a bounded claim by an unbounded one, and corrected a conclusion that a bounded run over a reduced alphabet had supported. We do not know how well this scales: it relies on a small, finite, observable state, and `withSomeSing` on every vertex requires a closed set of vertices.

Make the environment a first-class, reviewable object. The defect we found is in the environment. A typed state-machine library makes the machines safe; it does not make the *integration policy* between them safe. Writing that policy as a table of cues, holdings and releases, with the alternatives side by side, is what exposed it.

Did the functional style help? In three specific ways: pure machines made the oracle and the exploration trivial to run on any state; algebraic types made the vocabulary closed, so the typed boundary has no unreachable case; and `Feedback` as an ordinary value let us run one tick from an arbitrary state. The price was that observing a vertex inside `Basic`, whose

state type is existential, required wrapping the action (the observation is rebuilt with the same `AllowedTransition` proof, so no rule is rewritten).

7.2 Closing the gap: other techniques

Tying events to edges in the types is possible in principle with indexed monads [1] or dependently typed languages [2], and typestate [9] is the classic formulation of protocol-dependent types. We did not evaluate these alternatives; our claim is narrower: with the library as it is, a table and an oracle give a reviewable account of the event–destination relation, and the closed-loop properties need an exploration or a model checker whichever type system is used.

7.3 Threats to validity

Authorship. One person wrote the implementation, the table, the oracle and the TLA+ model. A misreading of the requirements could be in all of them. The table separates what the text says from what was assumed, but the review by the project's supervisor is pending and there is no third-party oracle.

The abstraction hypothesis. The "any length" claims depend on the loop's future being a function of the vertex triple. It holds by construction and was checked on 8 421 scenarios, but it is not proved.

Generalisation. There is one topology of four vertices and an environment with one slot. We did not vary topologies or scales, and the findings are about this model: the blind state is a property of the environment we wrote, not a defect of `crem` or of GHC. The domain has no real requirements behind it.

Mutants and tools. The mutants were chosen by the author. We used one version of each tool, one machine and one operating system. The shrinker finds local minima only.

Procedure. Every number comes from a recorded run, and the test suites include injected defects and hand-corrupted traces so that the checks demonstrably can fail.

8 Related work

This section is a pointed discussion, not a systematic review; a systematic search was planned (its protocol is in the artefact) but has not been performed, and no claim of novelty is made beyond the delimitation stated in the introduction.

Typed finite-state machines in Haskell have been explored through indexed monads [1], the `motor` library [11] and `crem` [8]; the latter lifts a topology to the type level with singletons [4], and its authors note that the topology is not tied to the inputs. Typestate [9] is the long-standing idea of types that change with protocol state. Property-based testing [3, 10] and model checking with TLA+ [6, 12] are the standard complements for behaviour that types do not cover; what we add is a concrete instance on a composed, fed-back typed machine, including a way to run the real machines from any state. The case of finite-state machines in pure functional languages was also treated by Malakhovski and Shalyto [7]; we cite it for the general idea only.

9 Conclusion

A type-level topology is a precise and useful guarantee, and it is a guarantee about edges. In a composed, fed-back system with an environment, the properties that matter are about sequences, and they can fail with every edge correct. In our case study the compiler rejected two of nine injected defects, a table-driven oracle and property-based tests caught the rest, and an exhaustive exploration of the real closed loop found a defect no edge-level check can see, which an independent TLA+ model reproduced. The reusable part is the method: a table with provenance, an oracle driven by it, a typed boundary, property-based tests, and closing the state graph by running the real machines from any vertex triple. What remains open is the independence of the specification from its author, other topologies and scales, and a systematic comparison with other approaches to the event–destination gap.

Data availability statement

The code, tables, scenarios, TLA+ specification, scripts and all logs are in the artefact of this work: a frozen snapshot of the `avionics-crem` repository (commit `9689ce8`) plus the package `avionics-validation`. **[Public repository URL and archived DOI to be added before submission; the repository was private at the time of writing.]** A single script (`run_all.sh`) rebuilds and reruns everything; `cabal.project.freeze` fixes the dependency versions.

Declaration of generative AI use

[To be defined with the external advisor.]

Funding and competing interests

[To be completed by the author.]

References

[1] Robert Atkey. 2009. Parameterised notions of computation. *Journal of Functional Programming* 19, 3–4 (2009), 335–376.

[2] Edwin Brady. 2013. Idris, a general-purpose dependently typed programming language: design and implementation. *Journal of Functional Programming* 23, 5 (2013), 552–593.

[3] Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*. ACM, New York, NY, USA, 268–279.

[4] Richard A. Eisenberg and Stephanie Weirich. 2012. Dependently typed programming with singletons. In *Proceedings of the 2012 Haskell Symposium (Haskell '12)*. ACM, New York, NY, USA, 117–130.

[5] Graham Hutton. 2016. *Programming in Haskell* (2nd ed.). Cambridge University Press, Cambridge, UK.

[6] Leslie Lamport. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, Boston, MA, USA.

[7] Yuri M. Malakhovski and Anatoly A. Shalyto. 2009. Konechnye avtomaty v chistykh funktsionalnykh yazykakh programmirovaniya [Finite state machines in pure functional programming languages]. *RSDN Magazine* 1 (2009). In Russian.

[8] Marco Perone and Georgios Karachalias. 2023. Crème de la Crem: Composable Representable Executable Machines (Architectural Pearl). arXiv:2307.09090 (presented at the FUNARCH workshop, ICFP 2023). Library: `crem`, https://hackage.haskell.org/package/crem.

[9] Robert E. Strom and Shaula Yemini. 1986. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171.

[10] The Hedgehog contributors. [n. d.]. Hedgehog: property-based testing for Haskell. https://github.com/hedgehogqa/haskell-hedgehog. Version 1.7 used in this work.

[11] The Motor contributors. [n. d.]. Motor: typed finite state machines in Haskell. https://github.com/owickstrom/motor.

[12] The TLA+ tools contributors. [n. d.]. The TLA+ tools (TLC model checker). https://github.com/tlaplus/tlaplus. Release v1.7.4 (TLC 2.19) used in this work.